Clojure Learning Journal

Episode 00 — The Curriculum & Learning Framework

Master reference document — read at the start of every project session

★ Updated: deps.edn Deep Dive added (W3D0) • Application Architecture added (W6D0)

1. Learner Profile

Understanding the learner shapes every design decision in this curriculum. This is not a generic Clojure course — it is built around one specific person.

Background	Python (primary), Lua (familiar), LISP (learning)
Learning style	Hands-on, project-driven. Learned Python best by writing a neural net for a master's thesis.
Motivation	Conciseness — same program in far fewer lines. Practical integration alongside Python, not replacement.
Reading	Paradigms of AI Programming (Norvig) — working through alongside this curriculum.
Schedule	30 min/day on weekdays. Longer, deeper sessions available on weekends.
Machines	Windows (WSL2 + native) now. Ubuntu machine arriving soon. Both will run identical setups.
Key insight	Must unlearn Python thinking first. The framework is designed so Claude drives the learning path, not the learner — avoiding the trap of learning Clojure through a Python lens.

2. Framework Philosophy

This curriculum is built on three principles that distinguish it from generic Clojure tutorials.

Principle 1 — Rewire, Don't Translate

The biggest mistake Python developers make when learning Clojure is translating Python patterns into Clojure syntax. Loops become loop/recur. Classes become records. Mutation becomes atoms. The result is Python written in parentheses — not Clojure.
Every session in Phase 1 is designed to break this habit by presenting Clojure on its own terms first, then showing the Python equivalent as contrast — never the other way around.

Principle 2 — REPL First, Always

In Python you write code then run it. In Clojure you evaluate expressions inside a live runtime. This is not a minor workflow difference — it is a fundamentally different relationship with code. Every session starts with the REPL open. Every concept is explored interactively before it is explained in prose.

Principle 3 — Claude Drives the Path

The learner identified a core problem: designing your own learning path means defaulting to what you already know. To solve this, Claude owns the curriculum design and session pacing. The learner shows up, engages, and asks questions. Claude ensures the sequence is correct and the thinking is truly Clojurian — not Pythonic.

> **The Norvig Connection**
> Paradigms of AI Programming is the ideal companion text because Norvig thinks in problems first, code second — exactly how Clojure should be approached. As the curriculum progresses into Phase 3, direct connections between PAIP patterns and Clojure idioms will be drawn explicitly.

Principle 4 — The Gap (Core Thread)

Every episode includes a dedicated section called 'The Gap' — the same problem solved in Python and Clojure, side by side. This is not a party trick. It is the core proof of why Clojure exists.

The Gap serves three purposes:

- Concrete evidence — you never take the language's power on faith. You see it every session.
- Correct motivation — understanding why fewer lines is possible forces you to understand the underlying concept (immutability, pure functions, the sequence abstraction). You cannot fake it.
- Progress marker — in Phase 1 the gap will sometimes feel small or even reversed. By Phase 2 it becomes dramatic. Watching it widen over weeks is one of the most motivating things in this curriculum.

> **Honest caveat**
> In Phase 1 the comparisons will occasionally feel unfair — Clojure will look strange and verbose where Python looks clean. This is normal and expected. The parentheses and prefix notation have a learning curve. The payoff arrives in Phase 2 when the gap becomes undeniable. Trust the Phase 1 strangeness — it is the rewiring working.

Example of what The Gap looks like — filter even numbers and double them:

Python (explicit loop):

```
result = []for n in numbers:    if n % 2 == 0:        result.append(n * 2)
```

Python (list comprehension):

```
result = [n * 2 for n in numbers if n % 2 == 0]
```

Clojure:

```
(->> numbers (filter even?) (map #(* 2 %)))
```

The Clojure version is one expression. It reads left to right as a pipeline. It creates no intermediate variables. It works on any sequence — not just lists. This is The Gap in its simplest form. By Phase 3 the examples will be far more dramatic.

3. The 3-Layer Approach

Phase	Name	Weeks	Focus	Goal
1	**Rewire**	1–2	Think in Clojure, not Python. Unlearn mutation and loops.	Evaluate expressions confidently. Understand nil, immutability, pure functions.
2	**Build**	3–5	Real mini-projects. Higher-order functions. Sequences.	Write idiomatic Clojure. Solve real problems with fewer lines than Python.
3	**Deepen**	6–8	Macros, state management, Java interop, PAIP tie-ins.	Understand what makes Clojure unique. Bridge to production patterns.

4. Full Curriculum Breakdown

Phase 1 — Rewire (Weeks 1–2)

Goal: stop thinking in Python. Start thinking in expressions, values, and transformations.

Week	Topic	Key Concepts	Weekend Bonus
W1D1	The REPL & expressions	s-expressions, prefix notation, everything returns a value	Explore 20+ expressions live. Notice what nil appears on and why.
W1D2	Data types	numbers, strings, booleans, nil, keywords, symbols	Compare to Python types. Find the surprises.
W1D3	Vectors & lists	[], '(), conj, first, rest, count	Build and destructure collections at the REPL.
W1D4	Maps & sets	{}, #{}, get, assoc, dissoc, keys, vals	Model a real Python dict problem in a Clojure map.
W1D5	def & defn	Defining values and functions. Docstrings.	Rewrite a Python function as a pure Clojure function.
W2D1	Pure functions	No side effects. Input → output. Referential transparency.	Find 3 Python functions you wrote that are impure. Rewrite them.
W2D2	if, cond, when	Conditional expressions. Everything is a value.	The 'no statements' concept — every branch returns something.
W2D3	let bindings	Local scope. Immutable locals. Destructuring basics.	Replace a Python block with a let form.
W2D4	map & filter	Transforming collections without loops.	Rewrite a Python list comprehension as map/filter.
W2D5	Review + mini project	Bring Phase 1 concepts together.	Build a small data transformation pipeline end-to-end.

Phase 2 — Build (Weeks 3–5)

Goal: write real, idiomatic Clojure. Solve problems you'd normally solve in Python.

> ★ **Curriculum Update**
> A dedicated deps.edn Deep Dive session (W3D0) has been added at the start of Phase 2, before the namespaces session (W4D1). This ensures you have a solid understanding of project configuration — dependencies, source paths, and aliases — before multi-file projects make it essential. Highlighted in yellow below.

Week	Topic	Key Concepts	Weekend Bonus
W3D0 ★ NEW	**deps.edn Deep Dive**	EDN format, source paths, dependencies, aliases. How the project config actually works.	Add a real dependency. Create a dev alias. Understand what Calva reads at jack-in.
W3D1	reduce & apply	Folding collections. Aggregation patterns.	Reimplement map and filter using reduce.
W3D2	Anonymous functions	#() syntax, fn forms, closures	Build a small function factory.
W3D3	Threading macros	-> and ->>. Reading pipelines left-to-right.	Refactor nested expressions into a clean pipeline.
W3D4	Lazy sequences	range, repeat, iterate, take, drop	Generate an infinite sequence. Don't crash.
W3D5	Recursion & loop/recur	Tail-call optimisation. When to use each.	Rewrite a Python recursive function.
W4D1	Namespaces	ns, require, use. Project structure.	Set up a proper multi-file project.
W4D2	Destructuring	Vectors, maps, nested. Function args.	Simplify a complex function signature.
W4D3	Sequences deep dive	seq abstraction, into, zipmap, group-by	Solve a data processing problem from scratch.
W4D4	Error handling	try/catch, ex-info, ex-data	Build a function with proper error propagation.
W4D5	Mini project 2	Data pipeline or small utility	Something you'd actually use.
W5D1 –5	Project week	Build something real with Phase 1+2 knowledge	A complete small program — chosen based on interest.

Phase 3 — Deepen (Weeks 6–8)

Goal: understand what makes Clojure genuinely powerful. Bridge to production and to PAIP.

> ★ **Curriculum Update**
> A dedicated Application Architecture session (W6D0) has been added at the start of Phase 3. It covers how real Clojure applications are layered — namespaces as boundaries, data flow across layers, and component systems (Integrant/Component) for wiring pieces together. Intentionally placed at Phase 3

entry: by W6 you have the vocabulary (namespaces, protocols, atoms, pure functions) to understand what the patterns mean. Highlighted in yellow below.

Week	Topic	Key Concepts	Weekend Bonus
W6D0 ★ NEW	Application Architecture in Clojure	Namespaces as layers. Data flow across boundaries. Component systems (Integrant/Component). How this differs from Python's class-based layering. The 'no objects' architecture.	Sketch the architecture of your Phase 2 project week app — redraw it the Clojure way.
W6D1	Atoms & state	atom, reset!, swap!. When mutation is acceptable.	Model a stateful counter the Clojure way.
W6D2	Refs & transactions	STM. dosync, ref, alter.	Why this matters for concurrency.
W6D3	Macros intro	defmacro, quote, syntax-quote, unquote	Write your first macro. Feel the power and the danger.
W6D4	Macros continued	Macro expansion. When to use vs functions.	PAIP connection: Norvig's macros in Common Lisp mapped to Clojure.
W6D5	Protocols & records	defprotocol, defrecord. Polymorphism without OOP.	Model a domain using records instead of classes.
W7D1	Java interop	. notation, new, import, type hints	Call a Java library from Clojure.
W7D2	core.async intro	go blocks, channels, <!, >!	Concurrency without threads.
W7D3	Transducers	Composable transformations. Performance.	Refactor a pipeline to use transducers.
W7D4	Spec & validation	clojure.spec, conform, explain	Add specs to a previous project.
W7D5	Performance basics	Benchmarking, type hints, persistent structures	Profile something. Fix it.
W8D1 –5	Capstone project	A real program combining all three phases	Something you're proud of. Potentially Python-interop or AI-adjacent.

5. Session Structure

Every chat in this project follows the same pattern regardless of topic. This consistency removes friction and maximises the 30 minutes.

Step	Time	What happens
1	Start	Claude reviews project .docx files to get up to speed on prior progress
2	Orient	Claude states the topic, what was covered last time, and what today builds on

3	Concept	Core concept introduced — always at the REPL first, explanation second
4	Demo	Interactive demo or exercise — learner types, evaluates, observes
5	The Gap	Same problem in Python vs Clojure — side by side. Conciseness made visible.
6	Bridge	Deeper Python/Lua contrast — why the gap exists, not just that it exists
7	Explore	Learner-driven questions, tangents, deeper dives (weekends get more of this)
8	Wrap	Summary of what was learned, what comes next
9	Doc	Claude produces Episode N .docx — learner adds it to project files

6. Environment Summary

Windows (Current — Native)

- Java: Temurin 21 LTS from adoptium.net
- Clojure CLI: version 1.12.4.1629 via win-install.ps1
- Editor: VS Code + Calva (217K installs) — Clojure & ClojureScript Interactive Programming
- Project root: E:\clojure_projects with deps.edn
- REPL: started via Ctrl+Shift+P → Calva: Start a Project REPL and Connect → deps.edn
- Evaluate expression: Alt+Enter with cursor on expression

Ubuntu (Coming Soon — Identical Setup)

- Java: Temurin 21 via adoptium apt repository
- Clojure CLI: linux-install.sh from github.com/clojure/brew-install
- Editor: VS Code native + Calva (same extension, no WSL layer needed)
- Sync: GitHub repo for code + VS Code Settings Sync for editor config

WSL2 Note
A network DNS issue was encountered when attempting Clojure setup inside WSL2 on Windows. This was bypassed by using the native Windows installation instead. The WSL2 fix (pointing resolv.conf to 8.8.8.8) can be attempted later if a Linux environment on the Windows machine is desired. For now, native Windows + the upcoming Ubuntu machine covers all needs.

7. Quick Reference — Commands & Shortcuts

REPL & Calva

Action	How

Start REPL	Ctrl+Shift+P → Calva: Start a Project REPL and Connect → deps.edn
Evaluate expression	Place cursor on expression → Alt+Enter
Evaluate file	Ctrl+Shift+P → Calva: Load/Evaluate Current File
Open Command Palette	Ctrl+Shift+P — search any action by name
Open Extensions	Ctrl+Shift+X
Start REPL (terminal)	cd to project folder → clj
Exit terminal REPL	Ctrl+D

Standing Reminder
30 minutes on weekdays — one concept, done well. Longer on weekends — go deeper, explore more, connect ideas. The REPL is always open. The strangeness you feel in Phase 1 is the learning working. Trust the process.